

Input request

```
curl --location 'https://api.groq.com/openai/v1/audio/speech' \  
--header 'Authorization: Bearer masked-token' \  
--header 'Content-Type: application/json' \  
--header 'Cookie: __cf_bm=fi.1Eqxv0WJ03G0nw61AWU5vBslC2hektfDcV99d_EQ-1784218327.703787-1.0.1.1-A-dirLmDt_rq7mTfbhbR.pshNCICqHOxumCj6.s.Zqqmz5P7hlymLG_dP700ekB1UK7SarxcTeTC6xTAWgg.DCRHa8CH.O_8M0QMTXKAlyU324OQvxje1x1zsebvLGchL' \  
--data '{  
  "model": "canopylabs/orpheus-v1-english",  
  "input": "[Cheerful] vanakkam. Good morining. How are you?",  
  "voice": "diana",  
  "response_format": "wav"  
}'
```

Output - Female Voice playback

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** `https://api.groq.com/openai/v1/audio/speech`
- Body (JSON):**

```
{  
  "model": "canopylabs/orpheus-v1-english",  
  "input": "[Cheerful] vanakkam. Good morining. How are you?",  
  "voice": "diana",  
  "response_format": "wav"  
}
```
- Status:** 200 OK
- Response Time:** 790 ms
- Response Size:** 203.33 KB
- Media Player:** A playback bar at the bottom shows the audio is at 0:00 / 0:04.

Input

```
curl --location 'https://api.groq.com/openai/v1/audio/speech' \
--header 'Authorization: Bearer masked-token' \
--header 'Content-Type: application/json' \
--header 'Cookie:
__cf_bm=fi.1Eqxv0WJ03G0nw61AWU5vBslC2hektfDcV99d_EQ-1784218327.703787-1.0.1.1-AdirLmDt
_rq7mTfbhbR.pshNCICqHOxumCj6.s.Zqqmz5P7hlymLG_dP700ekB1UK7SarxcTeTC6xTAWgg.DCRHa8CH.O_
8M0QMTXKAiyU3240Qvxje1xlzsebvLGchL' \
--data '{
  "model": "canopylabs/orpheus-arabic-saudi",
  "input": "[Cheerful] vanakkam. Good morining. How are you?",
  "voice": "fahad",
  "response_format": "wav"
}'
```

Output - Arabic male voice playback

The screenshot displays a REST client interface for a POST request to the endpoint `https://api.groq.com/openai/v1/audio/speech`. The request body is a JSON object with the following fields: `model` (canopylabs/orpheus-arabic-saudi), `input` ([Cheerful] vanakkam. Good morining. How are you?), `voice` (fahad), and `response_format` (wav). The response status is 200 OK, with a response time of 519 ms and a size of 135.83 KB. The response body is a WAV audio file, which is being played in a media player at the bottom of the interface. The media player shows a duration of 0:02 / 0:02.

GenAI-Jul-HomeAssignment / S01-Text to Audio

POST `https://api.groq.com/openai/v1/audio/speech` Send

Params Authorization Headers (13) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "model": "canopylabs/orpheus-arabic-saudi",
3   "input": "[Cheerful] vanakkam. Good morining. How are you?",
4   "voice": "fahad",
5   "response_format": "wav"
6 }
```

Body Cookies (1) Headers (22) Test Results 200 OK • 519 ms • 135.83 KB Save Response

Hex Preview Visualize

0:02 / 0:02

Input

```
curl --location 'https://api.groq.com/openai/v1/chat/completions' \
--header 'Content-Type: application/json' \
--header 'Authorization: Bearer masked token' \
--header 'Cookie:
__cf_bm=fi.1Eqxv0WJ03G0nw61AWU5vBsIC2hektfDcV99d_EQ-1784218327.703787-1.0.1.1-A
dirLmDt_rq7mTfbhbR.pshNCICqHOxumCj6.s.Zqqmz5P7hlymLG_dP700ekB1UK7SarxcTeTC6x
TAWgg.DCRHa8CH.O_8M0QMTXKAlyU324OQvxje1x1zsebvLGchL' \
--data '{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    {
      "role": "system",
      "content": "You are an API Specialist. Always respond in the same language as the
user."
    },
    {
      "role": "user",
      "content": "மிகவும் நன்றி! இந்த API-ஐ எப்படி பயன்படுத்துவது என்று
சொல்லுங்கள்."
    }
  ]
}'
```

Output - LLM responds in same language as user

The screenshot shows a REST client interface with the following details:

- URL:** `https://api.groq.com/openai/v1/chat/completions`
- Method:** POST
- Body:** A JSON object with a system message and a user message in Tamil.
- Response:** A 200 OK status with a JSON object containing an index, message, role, and content. The content is a detailed response in Tamil.

The response content (translated from Tamil) is as follows:

"வணக்கம்! இந்த API ஐ பயன்படுத்துவது எளிதான செயல்முறை. கீழே குறிப்பிடப்பட்டுள்ள படிகளை பின்பற்றவும்:\n\n1. **API பதிவு**: முதலில், நீங்கள் API க்கான பதிவு செய்ய வேண்டும். இது உங்களுக்கு ஒரு அடுத்தவைப்பு குறியீட்டை (API Key) வழங்கும்.\n\n2. **API ஆவணம்**: பதிவு செய்த பிறகு, உங்கள் API ஆவணத்தைப் பார்க்கவும். இது உங்களுக்கு API எந்திரவரையறைகள், முறைகள், மற்றும் பயன்பாட்டு விதிமுறைகளை விளக்கும்.\n\n3. **உங்கள் செயலி அல்லது வலைப்பக்கத்தை உருவாக்கவும்**: அடுத்தபடியாக, நீங்கள் உங்கள் செயலி அல்லது வலைப்பக்கத்தை உருவாக்கவும். இதில் நீங்கள் உங்கள் பயனர்களை உருவாக்கவும், தரவுகளைச் சேமிக்கவும், அல்லது ஏதேனும் செயல்பாடுகளைச் செய்யவும்.\n\n4. **API அமைப்புகளை வடிவமைக்கவும்**: பின்னர், நீங்கள்

POST S01-Text to Audio

POST S02- MultiLingual

+

GenAI-Jul-HomeAssignment / S02- MultiLingual

Save

Share

POST

https://api.groq.com/openai/v1/chat/completions

Send

Params

Authorization

Headers (11)

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Schema

Beautify

```
1 {
2   "model": "llama-3.3-70b-versatile",
3   "messages": [
4     {
5       "role": "system",
6       "content": "You are an API Specialist. Always respond in the same language as the user."
7     },
8     {
9       "role": "user",
10      "content": "सॉफ्टवेयर टेस्टिंग में AI का भविष्य क्या है?"
11    }
12  ]
13 }
```

Body

Cookies (1)

Headers (22)

Test Results

200 OK

3.22 s

2.5 KB

Save Response

JSON

Preview

Visualize

```
6   "choices": [
7     {
8       "index": 0,
9       "message": {
10        "role": "assistant",
11        "content": "सॉफ्टवेयर टेस्टिंग में एआई (आर्टिफिशियल इंटेलिजेंस) का भविष्य बहुत ही उज्ज्वल और रोमांचक है। एआई तकनीकें सॉफ्टवेयर टेस्टिंग प्रक्रिया में क्रांति ला रही हैं और इसे अधिक सटीक, कुशल और तेज बना रही हैं।\n\nएआई के उपयोग से सॉफ्टवेयर टेस्टिंग में कई फायदे हैं:\n1. **स्वचालित परीक्षण**: एआई विभिन्न प्रकार के सॉफ्टवेयर परीक्षणों को स्वचालित रूप से कर सकता है, जैसे कि यूजर इंटरफेस, एपीआई, और डेटाबेस परीक्षण।\n2. **परीक्षण डेटा विश्लेषण**: एआई परीक्षण डेटा का विश्लेषण करके खराबी की पहचान कर सकता है और समस्याओं को हल करने में मदद कर सकता है।\n3. **परीक्षण
```

Input

```
curl --location 'https://api.groq.com/openai/v1/chat/completions' \
--header 'Content-Type: application/json' \
--header 'Authorization: Bearer masked-token' \
--header 'Cookie:
__cf_bm=U7sVzXbbZObgL.i3xl8xJkZ9VAlel2SbRTt_A.ZCTrM-1784219245.1503613-1.0.1.1-Zj
HLWxjVpwwQhJweUjwwyEHnsULg7hPOGM6ajkA0.h3DmMcfghPZcmL7UurECQVeJOt6NKxox
D80hH4QTey28dS7RJuxUkXwZWG.GLr2TImRSjUCF33lcJq.W5xvIBEB' \
--data-raw '{
  "model": "openai/gpt-oss-120b",
  "messages": [
    {
      "role": "system",
      "content": "You are a Senior API Test Automation Architect with strong expertise in Java,
Rest Assured, TestNG, Maven, and API schema-driven test generation."
    },
    {
      "role": "user",
      "content": "I — INSTRUCTION\nGenerate Rest Assured Java code for Swagger Petstore
APIs.\n\nMANDATORY RULES:\n1) Generate ONLY Java Rest Assured code.\n2) Do NOT
include explanations, markdown, or extra text.\n3) Use TestNG as the test runner.\n4) Use
meaningful class, method, and variable names.\n5) Include imports required for Rest Assured,
TestNG, and assertions.\n6) Use reusable base URI setup.\n7) Cover CRUD operations for Pet
resource.\n8) Add assertions for status code and important response fields.\n9) Keep the code
clean, executable, and production-style.\n10) Do NOT use placeholder comments.\n\nC —
CONTEXT\nThe code will be used by QA engineers for API automation practice using Swagger
Petstore.\n\nP — PERSONA\nYou are a Senior API Test Automation Architect creating
enterprise-style reusable Rest Assured automation code.\n\nO — OUTPUT\nReturn ONLY
executable Java code with:\n- One test class\n- @BeforeClass setup\n- Test methods for
createPet, getPetById, updatePet, deletePet\n- Request body creation\n- Response
validation\n\nRules:\n- No markdown fences\n- No explanation text\n- No assumptions outside
Swagger Petstore Pet API\n- Use business-readable method names\n\nTARGET
API:\nSwagger Petstore\nResource: Pet\nOperations required:\n- Add a new pet\n- Find pet by
ID\n- Update an existing pet\n- Delete a pet"
    }
  ]
}'
```

Output

sends a prompt to the **Groq AI API** (using the OpenAI-compatible endpoint) and asks the **openai/gpt-oss-120b** model to generate **production-ready Java Rest Assured + TestNG CRUD test code** for the Swagger Petstore API.

The screenshot shows a REST client interface with a POST request to `https://api.groq.com/openai/v1/chat/completions`. The request body is raw text containing a system instruction and a user prompt. The system instruction is: `INSTRUCTION\ngenerate Rest Assured Java code for Swagger Petstore APIs.` The user prompt is: `\n\nMANDATORY RULES:\n(1) Generate ONLY Java Rest Assured code.\n(2) Do NOT include explanations, markdown, or extra text.\n(3) Use TestNG as the test runner.\n(4) Use meaningful class, method, and variable names.\n(5) Include imports required for Rest Assured, TestNG, and assertions.\n(6) Use reusable base URI setup.\n(7) Cover CRUD operations for Pet resource.\n(8) Add assertions for status code and important response fields.\n(9) Keep the code clean, executable, and production-style.\n(10) Do NOT use placeholder comments.\n\n- CONTEXT\nThe code will be used by QA engineers for API automation practice using Swagger Petstore.\n\n- PERSONA\nYou are a Senior API Test Automation Architect creating enterprise-style reusable Rest Assured automation code.\n\n- OUTPUT\nReturn ONLY executable Java code with:\n- One test class\n- @BeforeClass setup\n- Test methods for createPet, getPetById, updatePet, deletePet\n- Request body creation\n- Response validation\n\nRules:\n- No markdown fences\n- No explanation text\n- No`

The response is a JSON object with the following structure:

```
{
  "id": "chatcmpl-5b4dbfb9-0f8a-4e6a-9a3b-b0cdd50a7593",
  "object": "chat.completion",
  "created": 1784219304,
  "model": "openai/gpt-oss-120b",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "import io.restassured.RestAssured;\nimport io.restassured.http.ContentType;\n\nimport io.restassured.response.Response;\nimport org.testng.Assert;\nimport org."
      }
    }
  ]
}
```